

RESULTS AT A GLANCE

Bigger ≠ Better

Scaling model size often raises energy, latency, and VRAM faster than toy-task quality.

Quantization Changes Economics

q4/q8 variants can cut memory and energy enough to change where models can run.

No Single Winner

The best model depends on workload: code, summary, chat, RAG, or embeddings.

Hardware Depends on Scale

CPU can be fine for tiny local jobs; GPU wins when batching or concurrency grows.

OVERVIEW

Everyday LLM calls look simple from Python, but the real deployment decision is a tradeoff between **quality**, **latency**, **VRAM**, **energy**, and **cost**.

We benchmark chat completions, document summarization, classification-style prompts, RAG-style search, code generation, and batch embeddings across model sizes, quantization levels, architectures, and CPU/GPU configurations.

Thesis: the most accurate LLM is often not the most economically efficient one. LLM deployment is a systems engineering problem, not just model selection.

BENCHMARK SCOPE

19 models / variants	5 workload families	234 metric rows	L40S NVIDIA L40S
--------------------------------	-------------------------------	---------------------------	----------------------------

Question	Measured Signals	Why It Matters
Bigger model?	toy score, latency, joules, VRAM	find diminishing returns
Quantize?	q4/q8/fp16, tokens/J	fit smaller hardware
Which workload?	latency slices, joules, cost, score	avoid one-model thinking

MEASUREMENT PIPELINE

Each run produces one row of metrics: latency slices, token counts, active-device energy, memory, workload score, and request-cost estimates.

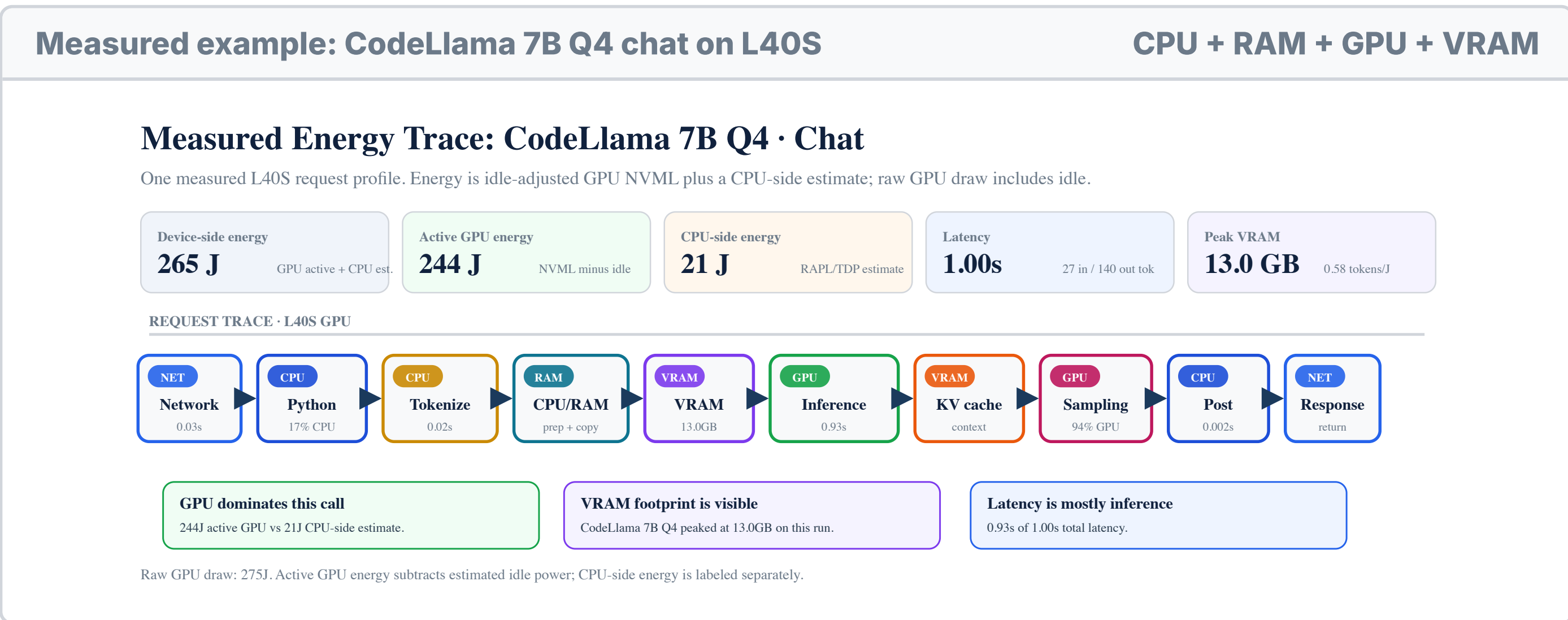
FIXTURES	Repeatable prompts and document sets for each workload.
CLIENT	Ollama/OpenAI-compatible adapter wraps model calls and response logs.
ENERGY	NVML samples GPU power/utilization/VRAM; CPU uses RAPL or labeled TDP estimate.
VISUALS	pandas + matplotlib generate latency waterfalls, cost bars, and active-energy figures.

Energy attribution matters: active-device energy is cleaner for model comparison; total wall energy is better for full-machine cost.

WHAT GETS TIMED

Slice	Meaning	Why Python Devs Care
Network	client call overhead	endpoint placement, batching
Tokenization	prompt processing / prefill	long docs are not free
Inference	model generation	model size dominates
Post-processing	parse, serialize, rank	pipeline glue is measurable

MEASURED ENERGY TRACE FOR ONE LLM



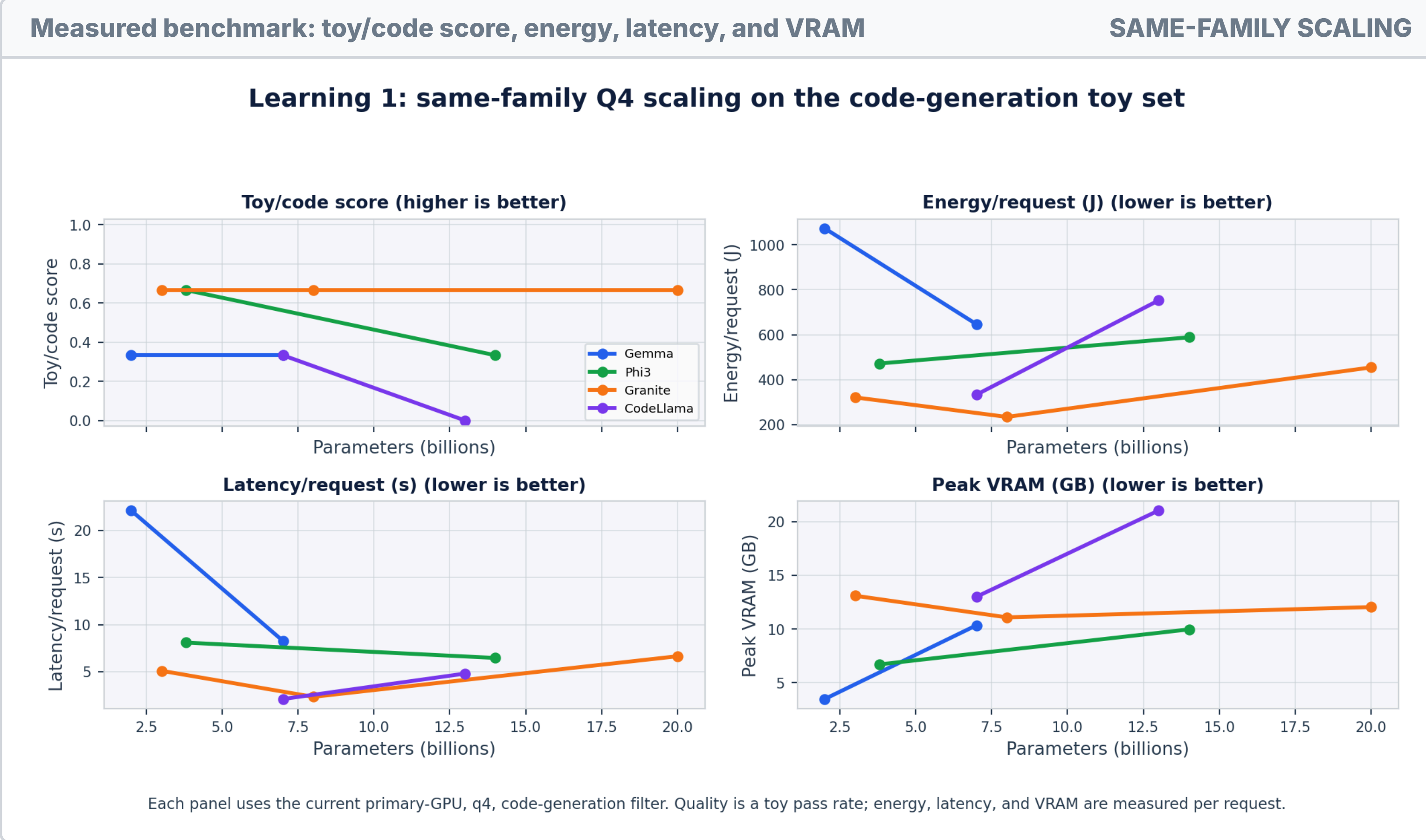
How to read it: one measured CodeLlama 7B Q4 chat request. The cards show active-device joules, latency, and peak VRAM; the trace locates Python client work, prefill, GPU inference, KV cache, sampling, and response parsing.

LEARNING 1: BIGGER MODELS HAVE DIMINISHING RETURNS

SAME ARCHITECTURE · DIFFERENT SIZES

Scaling parameters often increases infrastructure cost faster than model quality.

Same-family comparisons isolate parameter count: Gemma 2B/7B, Phi3 3.8B/14B, Granite 3B/8B/20B, CodeLlama 7B/13B where present.

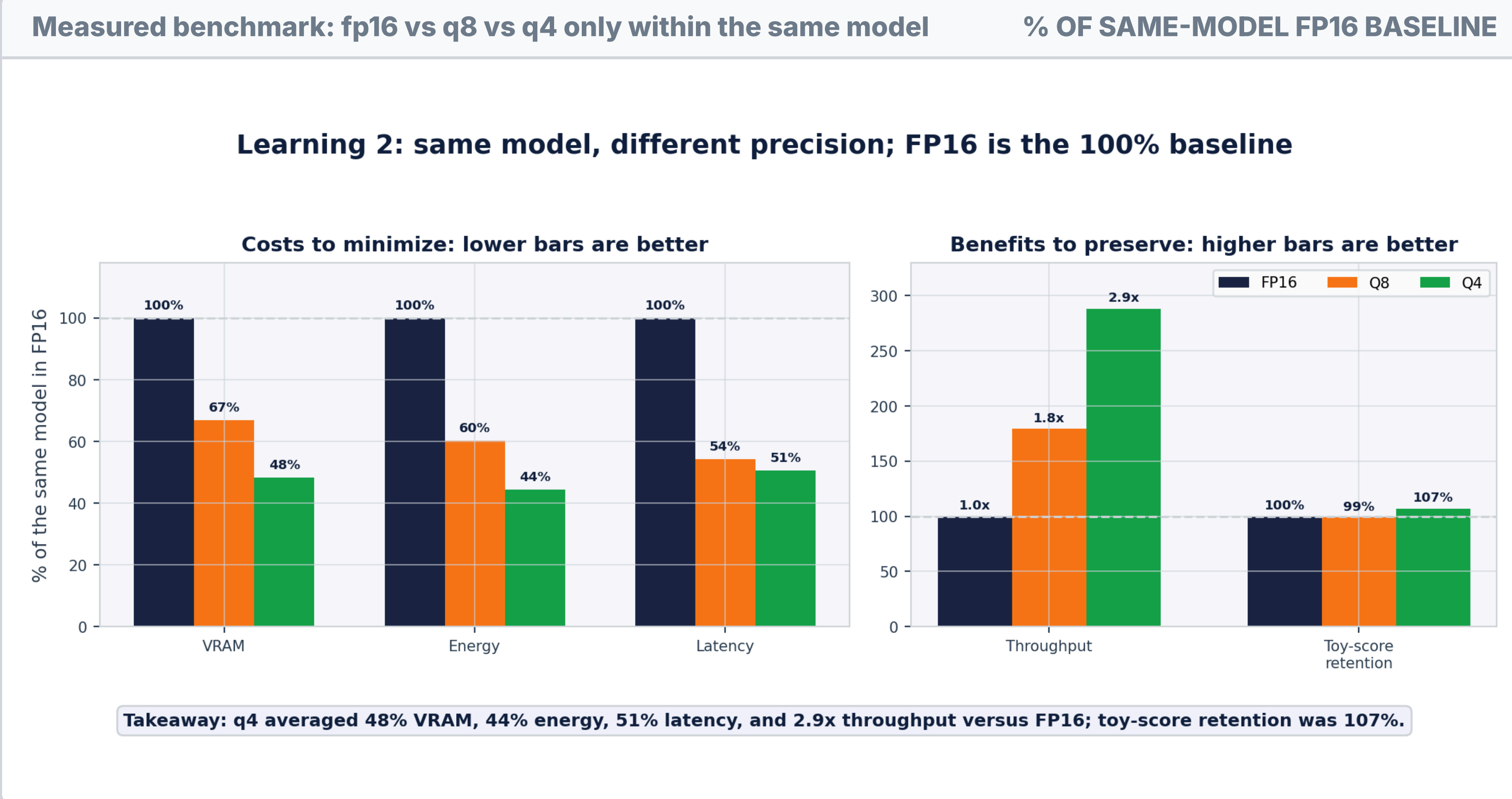


Score note: the quality panel is a tiny code-generation pass rate, not a claim that lower-parameter models are universally smarter. Developer lesson: bigger is not automatically cheaper, faster, more deployable, or more practical.

LEARNING 2: QUANTIZATION CHANGES THE ECONOMICS OF AI

SAME MODEL · DIFFERENT PRECISION

4-bit and 8-bit models often retain useful quality while reducing memory, energy, and latency.

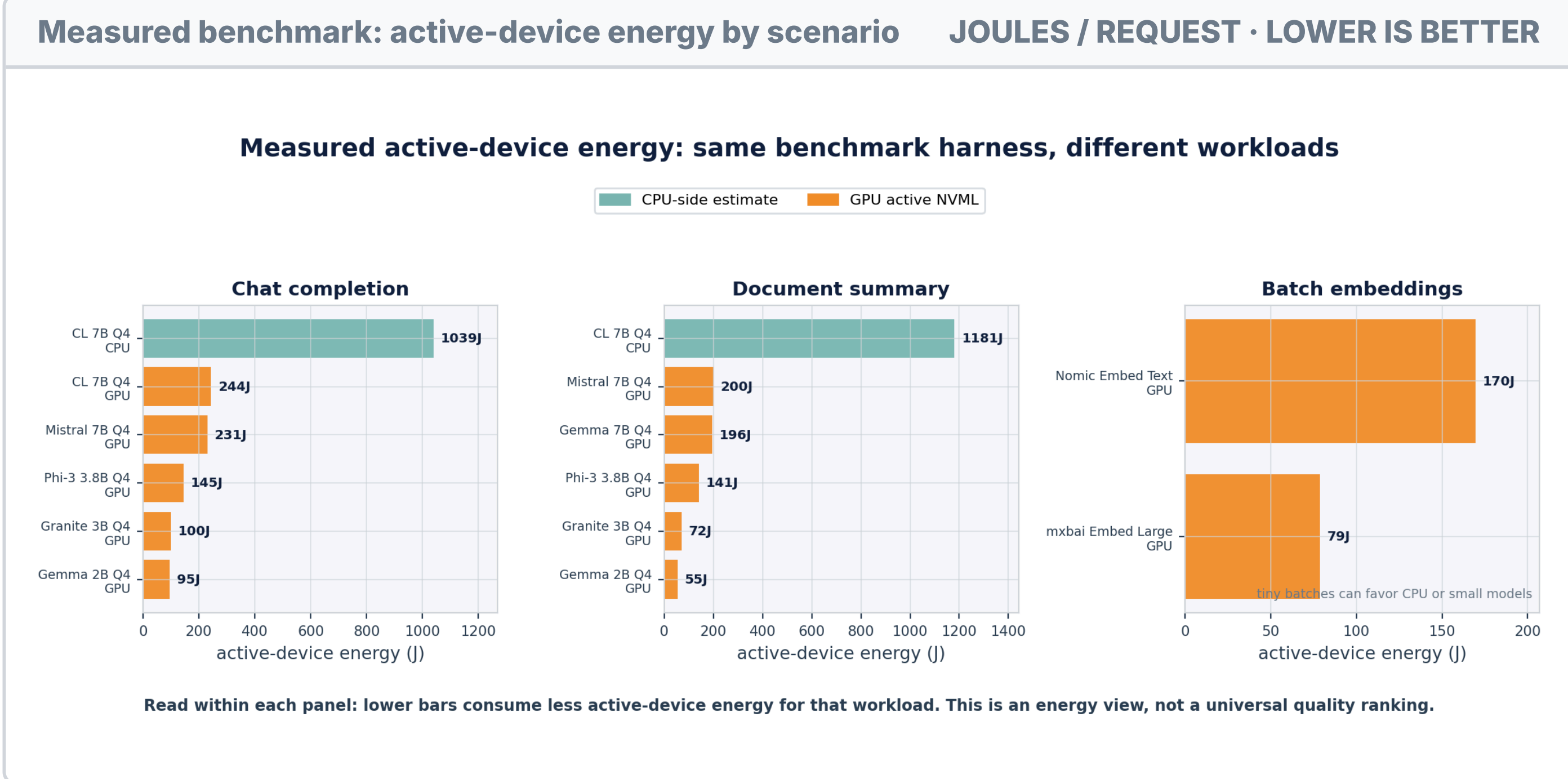


How to read it: FP16 is the 100% baseline for the exact same model; q8/q4 are compressed representations. Left: costs to minimize (VRAM, energy, latency). Right: benefits to preserve or improve (throughput, toy-score retention). Bars average only complete same-model FP16/q8/q4 triplets.

LEARNING 3: ENERGY DEPENDS ON THE WORKLOAD

COMMON WORKLOADS · ACTIVE-DEVICE JOULES

Chat, document summaries, RAG, and embeddings do not stress the same parts of the stack.

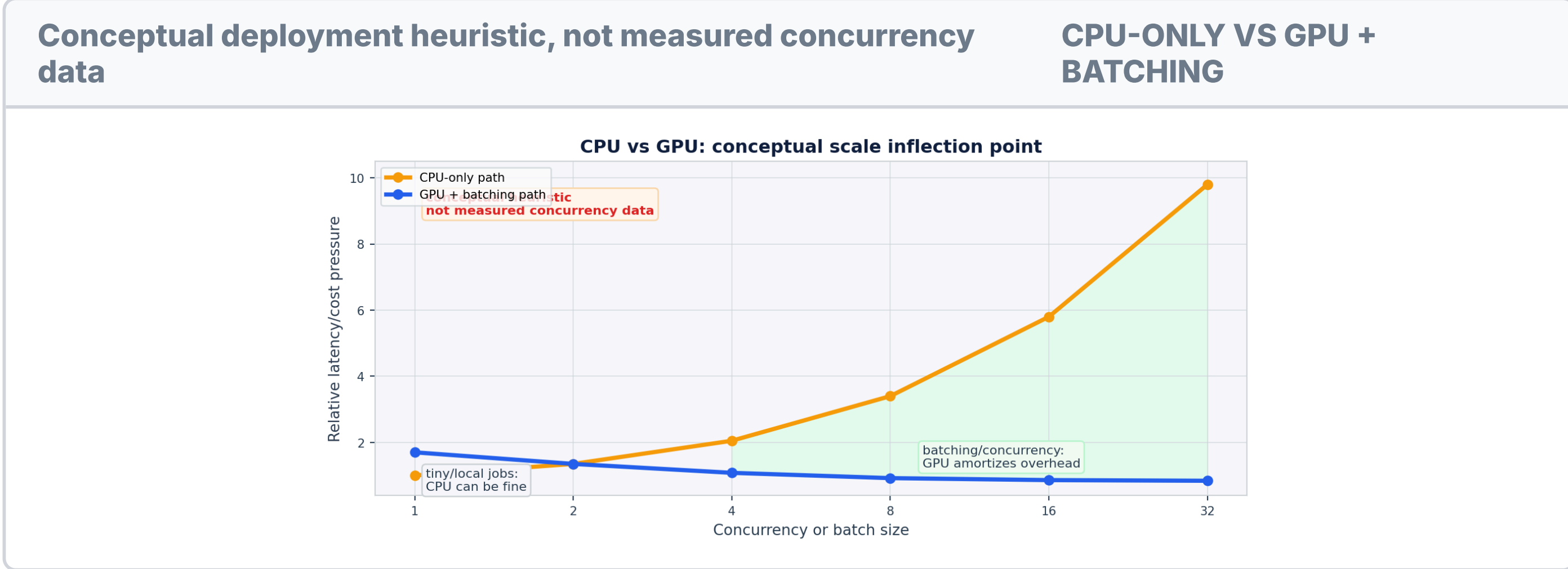


How to read it: each panel is a different workload. A model/hardware pair that is cheap for chat may not be cheap for long summaries or embedding batches, so the benchmark must be repeated per scenario.

CPU VS GPU: THE SCALE INFLECTION POINT

HARDWARE SCALE · CONCEPTUAL HEURISTIC

Hardware choice changes when prompts, outputs, batching, or concurrency grow. Single-request latency is not the same as throughput.



CPU is reasonable for

- prototyping
- local scripts
- tiny embedding jobs
- single-user offline tools
- low request volume

GPU becomes better for

- long prompts / outputs
- batch embeddings
- multi-user chat
- latency-sensitive services
- high concurrency

Tiny embedding jobs may not amortize GPU overhead. Measure batch size before assuming the GPU path wins.

PRACTICAL HEURISTICS

#	Heuristic
1	Benchmark the workload, not the model card.
2	If quality barely improves but joules rise, use the smaller model.
3	If VRAM is the deployment constraint, use q4/q8.
4	If the workload changes, re-benchmark; do not reuse one winner.
5	If concurrency grows, use GPU batching.
6	If the workload is tiny and local, CPU may be good enough.
7	Track tokens/joule, not just accuracy.